Basic Instructions for Downloading Python Libraries

Python libraries can be installed using the pip command in Command Prompt or the terminal.

Steps to install a Python library

1. Open Command Prompt or the terminal in VS Code.
2. Check whether Python is installed by typing:

```
python --version
```

3. Install the required library using pip.

```
pip install library_name
```

4. Import the library into your Python program and use it.

Example: Installing Pandas

```
pip install pandas
```

Practical 1: Collect data from a real-life business scenario and perform exploratory data analysis (EDA) to gain insights into the dataset.

Libraries required: pandas, matplotlib, seaborn

```
# Practical 1: Exploratory Data Analysis (EDA)

# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# To set maximum rows to display
# pd.set_option('display.max_rows', 500)

# Step 2: Load Dataset
df = pd.read_csv("sales.csv")

print("Display All Records in String Format")
print(df.to_string())

# Step 3: Display First Five and Last Ten Records
print("\nDisplay First Five Records")
print(df.head())

print("\nDisplay Last Ten Records")
print(df.tail(10))

# Step 4: Check Dataset Information
print("\nDisplay Dataset Information")
df.info()
```

```
27.
28. # Step 5: Statistical Summary
29. print("\nDisplay Statistical Summary")
30. print(df.describe())
31.
32. # Step 6: Check Missing Values
33. print("\nCheck Missing Values")
34. print(df.isnull().sum())
35.
36. # Step 7: Check Duplicate Records
37. print("\nCheck Duplicate Records")
38. print(df.duplicated().sum())
39.
40. # Step 8: Check Product Distribution
41. print("\nDisplay Product Distribution")
42. print(df["Product"].value_counts())
43.
44. sns.countplot(data=df, x="Product")
45. plt.xticks(rotation=270)
46. plt.title("Product Distribution || Ayesha Sarguru")
47. plt.xlabel("Product")
48. plt.ylabel("Count")
49. plt.tight_layout()
50. plt.show()
51.
52. # Step 9: Gender Distribution
53. sns.countplot(data=df, x="Gender")
54. plt.title("Gender Distribution || Ayesha Sarguru")
55. plt.xlabel("Gender")
56. plt.ylabel("Count")
57. plt.tight_layout()
58. plt.show()
59.
60. # Step 10: City-wise Customers
61. sns.countplot(data=df, x="City")
62. plt.xticks(rotation=45)
63. plt.title("City-wise Customers || Ayesha Sarguru")
64. plt.xlabel("City")
65. plt.ylabel("Number of Customers")
66. plt.tight_layout()
67. plt.show()
68.
69. # Step 11: Price Distribution
70. plt.hist(df["Price"], bins=4)
71. plt.xlabel("Price")
72. plt.ylabel("Frequency")
73. plt.title("Price Distribution || Ayesha Sarguru")
74. plt.tight_layout()
75. plt.show()
76.
77. # Step 12: Box Plot of Total Sales
78. sns.boxplot(y=df["Total"])
79. plt.title("Box Plot of Total Sales || Ayesha Sarguru")
80. plt.ylabel("Total Sales")
81. plt.tight_layout()
82. plt.show()
83.
84. # Step 13: Correlation Heatmap
85. sns.heatmap(
86.     df.corr(numeric_only=True),
87.     annot=True,
88.     cmap="coolwarm"
89. )
90. plt.title("Correlation Heatmap || Ayesha Sarguru")
91. plt.tight_layout()
92. plt.show()
93.
94.
```

Practical 2: Use the Central Limit Theorem to analyze the sampling distribution of a sample mean and estimate population parameters.

Libraries required: numpy, matplotlib

```python
import numpy as np
import matplotlib.pyplot as plt

# Set seed
np.random.seed(42)

# Create population
population = np.random.exponential(
    scale=3500,
    size=100000
)

# Sample size
sample_size = 300

# Number of samples
number_of_samples = 1000

# Store sample means
sample_means = []

# Generate samples
for i in range(number_of_samples):

    sample = np.random.choice(
        population,
        size=sample_size,
        replace=False
    )

    sample_means.append(sample.mean())

sample_means = np.array(sample_means)

# Calculate statistics
population_mean = population.mean()
sampling_mean = sample_means.mean()
standard_error = sample_means.std(ddof=1)

print("Population Mean:", round(population_mean, 2))
print("Mean of Sample Means:", round(sampling_mean, 2))
print("Standard Error:", round(standard_error, 2))

# Visualize the Sampling Distribution
plt.figure(figsize=(8, 5))

plt.hist(
    sample_means,
    bins=30
)

plt.title("Sampling Distribution of Sample Mean")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")

plt.show()

```

Practical 3 Aim: To calculate descriptive statistics (mean, median, mode, variance, standard deviation and range) for a business variable of interest.

Libraries required: pandas, matplotlib, seaborn

```
# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load the Dataset
df = pd.read_csv("sales.csv")

# Step 3: Display First Five Records
print("Display First Five Records")
print(df.head())

# Step 4: Select the Variable
sales = df["Total"]
print("Selected Variable : Total - Sales")
print(sales)

# Calculate Descriptive Statistics
# Step 5: Display Summary Statistics
print("Display Summary Statistics")
print(sales.describe())

# Step 6: Calculate Mean
print("Mean:")
print(sales.mean())

# Step 7: Calculate Median
print("Median")
print(sales.median())

# Step 8: Calculate Mode
print("Mode:")
print(sales.mode())

# Step 9: Calculate Variance
print("Variance:")
print(sales.var())

# Step 10: Calculate Standard Deviation
print("Standard Deviation:")
print(sales.std())

# Step 11: Calculate Minimum and Maximum
print("Minimum Sales :", sales.min())
print("Maximum Sales :", sales.max())

# Step 12: Calculate Range
print("Range:")
print(sales.max() - sales.min())

```

Practical 4 Aim: To calculate confidence intervals for population means and proportions using sample data and interpret the results in a business context.

```
import math

# Sample information
sample_mean = 3520
population_std = 800
sample_size = 50

# Z-value for 95% confidence level
z = 1.96

# Calculate standard error
standard_error = population_std / math.sqrt(sample_size)

# Calculate margin of error
margin_of_error = z * standard_error

# Calculate confidence interval
lower_limit = sample_mean - margin_of_error
upper_limit = sample_mean + margin_of_error

print("Sample Mean:", sample_mean)
print("Standard Error:", round(standard_error, 2))
print("Margin of Error:", round(margin_of_error, 2))

print(
    "95% Confidence Interval:",
    round(lower_limit, 2),
    "to",
    round(upper_limit, 2)
)


# confidence intervals for proportions

# Survey information
n = 500
satisfied = 325

# Sample proportion
p = satisfied / n

# Z-value for 95% confidence
z = 1.96

# Standard error
standard_error = math.sqrt(
    p * (1 - p) / n
)

# Margin of error
margin_of_error = z * standard_error

# Confidence interval
lower = p - margin_of_error
upper = p + margin_of_error

print("Sample Proportion:", round(p, 4))
print("Sample Proportion (%):", round(p * 100, 2))

print("Standard Error:", round(standard_error, 4))
print("Margin of Error:", round(margin_of_error, 4))

```

```
print(
    "95% Confidence Interval:",
    round(lower * 100, 2),
    "% to",
    round(upper * 100, 2),
    "%"
)

```

Practical 5 Aim: To visualize the distribution of a business variable using histograms, box plots and violin plots and to identify outliers.

Libraries required: pandas, matplotlib, seaborn

```
# Variables of Interest
# We will analyze the Total column.

# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load the Dataset
df = pd.read_csv("sales.csv")

# Step 3: Display First Five Records
print("Display First Five Records")
print(df.head())

# Step 4: Select the Variable
sales = df["Total"]
print("Selected Variable : Total - Sales")
print(sales)

# Visualize the Distribution

# Step 5: Histogram
plt.figure(figsize=(8,5))
plt.hist(sales, bins=10, edgecolor="black")
plt.title("Distribution of Total Sales")
plt.xlabel("Total Sales")
plt.ylabel("Frequency")
plt.show()

# Step 6: Histogram with KDE Curve
plt.figure(figsize=(8,5))
sns.histplot(sales, bins=10, kde=True)
plt.title("Histogram of Total Sales")
plt.show()

# The KDE (Kernel Density Estimate) curve provides a smoother view of the data distribution.

# Step 7: Box Plot
plt.figure(figsize=(6,4))
sns.boxplot(y=sales)
plt.title("Box Plot of Total Sales")
plt.show()

# Step 8: Violin Plot (Optional)
plt.figure(figsize=(6,4))
sns.violinplot(y=sales)
plt.title("Violin Plot of Total Sales")
plt.show()

# Step 9: Identify Outliers
```

```
Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)

IQR = Q3 - Q1

Lower = Q1 - 1.5 * IQR
Upper = Q3 + 1.5 * IQR
print("Lower outlier value:", Lower)

print("Upper outlier value:", Upper)
outliers = df[(sales < Lower) | (sales > Upper)]
print("outliers")
print(outliers)

# Outliers represent unusually high or low transactions that may require further
# investigation.

```

Practical 6 Aim: To apply bootstrapping to estimate confidence intervals for non-parametric statistics such as the median.

Libraries required: numpy

```
import numpy as np

# Original sample
data = np.array([
    450, 520, 600, 480, 750,
    900, 550, 620, 1500, 680,
    720, 500, 580, 810, 650
])

# Number of bootstrap samples
B = 10000

# Store bootstrap medians
bootstrap_medians = []

# Set seed for reproducibility
np.random.seed(42)

# Bootstrap procedure
for i in range(B):

    # Sample with replacement
    bootstrap_sample = np.random.choice(
        data,
        size=len(data),
        replace=True
    )

    # Calculate median
    median_value = np.median(bootstrap_sample)

    bootstrap_medians.append(median_value)

# Convert to NumPy array
bootstrap_medians = np.array(bootstrap_medians)

# Calculate original sample median
original_median = np.median(data)

# Calculate 95% confidence interval
lower = np.percentile(
    bootstrap_medians,
```

```
    2.5
)

upper = np.percentile(
    bootstrap_medians,
    97.5
)

print("Original Sample Median:", original_median)

print(
    "95% Bootstrap Confidence Interval:",
    lower,
    "to",
    upper
)

```

Practical 7 Aim: To perform correlation analysis to determine the strength and direction of relationships between advertising channels and sales revenue.

Libraries required: pandas, matplotlib, seaborn

```


# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load Dataset
df = pd.read_csv("advertising_sales Pract3.csv")


# Step 3: Display Dataset
print("Display Dataset : head")
print(df.head())


# Step 4: Check Dataset Information
print("Check Dataset Information")
print(df.info())


# Step 5: Calculate Correlation Matrix
correlation = df[['TV_Advertising',
                  'Radio_Advertising',
                  'Newspaper_Advertising',
                  'Sales_Revenue']].corr()

print(correlation.to_string())


# Step 6: Display Correlation Heatmap
plt.figure(figsize=(8,6))

sns.heatmap(correlation,
            annot=True,
            cmap="coolwarm",
            linewidths=0.5)

plt.title("Correlation Matrix")

```

```
plt.show()

# Step 7: Scatter Plot (TV Advertising vs Sales Revenue)
plt.figure(figsize=(7,5))

sns.scatterplot(data=df,
                x="TV_Advertising",
                y="Sales_Revenue")

plt.title("TV Advertising vs Sales Revenue")

plt.show()

```

Practical 8 Aim: To formulate null and alternative hypotheses related to a business problem, conduct a hypothesis test using appropriate statistical tests, and interpret the results.

Libraries required: numpy, scipy

```
import numpy as np
from scipy import stats

# Sales data after implementing the new marketing strategy
sales = np.array([
    52000, 54000, 51000, 55000, 53000,
    56000, 52000, 57000, 54000, 55000,
    53000, 58000, 51000, 56000, 54000,
    57000, 55000, 53000, 59000, 56000
])

# Hypothesized population mean
population_mean = 50000

# Perform one-sample t-test
t_statistic, p_value_two_tailed = stats.ttest_1samp(
    sales,
    population_mean
)

# Convert two-tailed p-value to one-tailed p-value
if t_statistic > 0:
    p_value = p_value_two_tailed / 2
else:
    p_value = 1 - (p_value_two_tailed / 2)

print("Sample Mean:", round(np.mean(sales), 2))
print("t-statistic:", round(t_statistic, 4))
print("One-tailed p-value:", round(p_value, 4))

alpha = 0.05

if p_value < alpha:
    print("Decision: Reject H0")
    print("Conclusion: There is significant evidence that sales increased.")
else:
    print("Decision: Fail to reject H0")
    print("Conclusion: There is insufficient evidence that sales increased.")

```

Practical 9 Aim: To simulate a probability experiment, such as rolling a die, using Python and calculate the probabilities of different outcomes.

```
import random
from collections import Counter

# Number of times the die is rolled
n = 10000

# Simulate rolling a fair die
rolls = [random.randint(1, 6) for _ in range(n)]
print(rolls)

# Count occurrences of each outcome
counts = Counter(rolls)
print(counts)
print("Outcome\tFrequency\tExperimental Probability")

for outcome in range(1, 7):
    frequency = counts[outcome]
    probability = frequency / n
    print(outcome, "\t", frequency, "\t\t", round(probability, 4))

# Theoretical probability
print("\nTheoretical Probability of each outcome =", round(1/6, 4))

```

Practical 10 Aim: To generate random numbers from Normal, Uniform, and Exponential probability distributions using Python and analyze their properties.

Libraries required: numpy, matplotlib

```
import numpy as np
import matplotlib.pyplot as plt

# Number of random observations
n = 10000

# Generate random numbers

# Normal distribution
normal_data = np.random.normal(
    loc=50,
    scale=10,
    size=n
)

# Uniform distribution
uniform_data = np.random.uniform(
    low=0,
    high=100,
    size=n
)

# Exponential distribution
exponential_data = np.random.exponential(
    scale=10,
    size=n
```

```
)

# Display first 10 values
print("Normal Distribution:")
print(normal_data[:10])

print("\nUniform Distribution:")
print(uniform_data[:10])

print("\nExponential Distribution:")
print(exponential_data[:10])

# Calculate properties
print("\nStatistical Properties")

print("\nNormal Distribution")
print("Mean:", np.mean(normal_data))
print("Standard Deviation:", np.std(normal_data))

print("\nUniform Distribution")
print("Mean:", np.mean(uniform_data))
print("Standard Deviation:", np.std(uniform_data))

print("\nExponential Distribution")
print("Mean:", np.mean(exponential_data))
print("Standard Deviation:", np.std(exponential_data))

# Visualizing the Distributions

# Normal Distribution
plt.figure(figsize=(8, 5))
plt.hist(normal_data, bins=30)
plt.title("Normal Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

# Uniform Distribution
plt.figure(figsize=(8, 5))
plt.hist(uniform_data, bins=30)
plt.title("Uniform Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

# Exponential Distribution
plt.figure(figsize=(8, 5))
plt.hist(exponential_data, bins=30)
plt.title("Exponential Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

```

Practical 11 Aim: Develop a decision tree model to make business decisions considering uncertainties and associated probabilities at each decision point.

Libraries required: matplotlib

```
# Decision Tree: Product Selection

# Product A
p_high_A = 0.60
p_low_A = 0.40

profit_high_A = 80000
profit_low_A = -20000

# Product B
p_high_B = 0.70
p_low_B = 0.30

profit_high_B = 60000
profit_low_B = 10000

# Display decision tree information

print("DECISION TREE")
print("=======================")

print("\nProduct A")
print(" High Demand:", p_high_A, "-> Rs", profit_high_A)
print(" Low Demand :", p_low_A, "-> Rs", profit_low_A)

print("\nProduct B")
print(" High Demand:", p_high_B, "-> Rs", profit_high_B)
print(" Low Demand :", p_low_B, "-> Rs", profit_low_B)

# Expected Monetary Value Calculation

# Product A
ev_A = (p_high_A * profit_high_A) + \
       (p_low_A * profit_low_A)

# Product B
ev_B = (p_high_B * profit_high_B) + \
       (p_low_B * profit_low_B)

print("\nEXPECTED MONETARY VALUE")
print("=======================")

print("Expected Value of Product A: Rs", ev_A)
print("Expected Value of Product B: Rs", ev_B)

# Select the optimal decision
if ev_A > ev_B:
    print("\nOptimal Decision: Select Product A")
elif ev_B > ev_A:
    print("\nOptimal Decision: Select Product B")
else:
    print("\nBoth alternatives have the same Expected Value")

import matplotlib.pyplot as plt

products = ["Product A", "Product B"]
expected_values = [ev_A, ev_B]

plt.figure(figsize=(7, 5))
plt.bar(products, expected_values)

plt.title("Expected Monetary Value Comparison")
```

```
63.
64. plt.xlabel("Business Alternatives")
65. plt.ylabel("Expected Value (Rs)")
66.
67. plt.show()
68.
69.
70.
```

Practical 12 Aim: Apply the concept of expected value to evaluate different decision alternatives and select the optimal one.

Libraries required: matplotlib

```
 1. # Expected Value Calculation
 2.
 3. # Product A
 4. p_high_A = 0.60
 5. p_low_A = 0.40
 6.
 7. profit_high_A = 80000
 8. profit_low_A = -20000
 9.
10. # Product B
11. p_high_B = 0.70
12. p_low_B = 0.30
13.
14. profit_high_B = 60000
15. profit_low_B = 10000
16.
17. # Calculate Expected Value
18. ev_A = (p_high_A * profit_high_A) + (p_low_A * profit_low_A)
19.
20. ev_B = (p_high_B * profit_high_B) + (p_low_B * profit_low_B)
21.
22. # Display Expected Values
23. print("Expected Value of Product A:", ev_A)
24. print("Expected Value of Product B:", ev_B)
25.
26. # Select the Best Product
27. if ev_A > ev_B:
28.     print("Best Decision: Launch Product A")
29. else:
30.     print("Best Decision: Launch Product B")
31.
32.
```